MVP 5 is the communications layer: Photon texts the owner, receives their decision, and—only after approval—connects an automated call to the technician. Technician responses update the same work order shown on the dashboard.

For the simplest complete version, make the technician call a **spoken notification**, then collect acceptance and repair findings by text. An interactive Nemotron phone conversation is a stretch goal.

1. Important Photon implementation detail

This specification assumes **Photon's Stable Spectrum platform**, matching our earlier `spectrum-ts` plan.

Photon also has a separate Beta platform with different credentials and API contracts. Before coding, confirm which project the hackathon account provides. Don't mix Stable SDK examples with Beta credentials. Photon's version guide

For Stable Spectrum:

- Text messaging uses the configured messaging provider.
- Live phone calls connect through a **SIP application**.
- The current documentation lists messaging-SDK call control as not yet available.
- Sending a voice note is different from placing a phone call. Spectrum Voice documentation

Therefore, don't assume a method such as `photon.callTechnician()` already exists. That would be **our application function**, implemented using the configured calling system.

2. What this MVP must accomplish

The required communication loop is:

1. Receive a communication request from the backend/Nemotron workflow.
2. Text the owner with a clarification or approval request.
3. Receive and identify the owner's reply.
4. Submit the decision to the backend.
5. After backend authorization, initiate a technician call.
6. Play the approved work-request summary.
7. Receive explicit technician acceptance or refusal.
8. Collect progress and repair findings.
9. Send useful owner updates.
10. Preserve communication references and outcomes.

Use teammates as the registered owner and technician during the demonstration.

3. Responsibilities and boundaries

Component	Responsibility
Nemotron — MVP 4	Investigate, formulate questions, propose work, select an eligible technician, and request communication actions.
Backend — MVP 2	Own incidents, proposals, approvals, work orders, permissions, and action deduplication.
Photon integration — MVP 5	Send/receive texts and connect approved calls through the calling adapter.
Frontend — MVP 3	Display requests, owner decisions, and recorded communication progress.

MVP 5 must not maintain a separate approval database or independently decide that a technician visit is authorized.

It receives an action, executes the permitted communication, and returns the result.

4. Minimum setup

Requirement	What to establish
Photon project	A working sponsor-provided or otherwise configured project.
Text channel	One channel that can actually reach the owner and receive replies.
Calling line	A line enabled for the selected Photon voice setup.
Credentials	Stored in backend environment configuration.
Owner contact	A known, registered demo recipient.
Technician contact	A known, registered demo recipient.
SIP/voice runner	A compatible application capable of initiating calls and playing audio.
Event reception	A working message stream or verified webhook path.

For Stable Spectrum, outbound calling requires a project-owned iMessage line and a compatible SIP application. WhatsApp numbers cannot serve as the Spectrum calling line. [Outbound-call prerequisites]

Do not assume that successfully sending a text proves calling is configured.

5. Recommended architecture

Use two adapters behind one communications interface:

Adapter	Purpose
Messaging adapter	Owner questions, approval requests, replies, technician texts, and owner updates.
Calling adapter	Start an approved outbound call, play the work summary, and report call outcomes.

The flow is:

Nemotron → backend communication action → messaging or calling adapter → Photon → recipient

Responses return through:

Photon/SIP events → normalized communication event → backend → resumed Nemotron workflow → dashboard

Run these modules alongside the existing Node backend where practical. A separate voice runner may be necessary for the SIP connection.

6. Prove calling before building the full workflow

The calling developer should first establish:

1. A call reaches the teammate's phone.
2. The expected caller ID appears.
3. Audio can be heard.
4. The call can be initiated programmatically.
5. Completion or failure can be reported back to the backend.

A manually dialed softphone call is useful for setup testing, but it does not complete the automated application workflow.

For Stable Spectrum, the documented outbound configuration includes:

Setting	Value
SIP host	`sip.spectrum.photon.codes`
Server port	`5061`

Transport	TLS
Username	Spectrum project ID
Password	Spectrum project secret
Registration	Off; use a compatible no-registration/trunk setup
Caller ID	A line owned by the project

Use a working sponsor example or compatible SIP application rather than writing the SIP protocol yourself. Official outbound setup

The chosen application must support audio playback and programmatic control—not merely display a dial pad.

7. Owner clarification messages

Nemotron may need information before recommending service.

For example, an office light reading drops. The agent may ask whether the sensor is covered.

The communication action should include:

- Incident ID.
- Owner contact reference.
- Question ID.
- Question text.
- Expected response category.
- Creation time.

Keep this separate from approval.

Owner reply	Interpretation
“Yes, something was covering it.”	Answer to a clarification question.
“No, nothing is covering it.”	Answer to a clarification question.
“Approve SR-104.”	A decision about the identified service request.

A “yes” answering a sensor question must never accidentally authorize a technician visit.

After receiving clarification, update the backend and resume the agent. The agent decides whether readings should be checked again or service recommended.

8. Owner approval request

Send a short message containing:

Information	Purpose
Building	Identify where the issue is.
Observation	Explain what was detected.
Uncertainty	Avoid presenting a suspected cause as confirmed.
Proposed action	Explain what approval authorizes.
Request identifier	Match the reply to the correct proposal.
Response instructions	Make approval and decline unambiguous.

An illustrative message would explain that office light levels remain low, the cause is unconfirmed, and a maintenance inspection is recommended. It would ask the owner to reply:

- `APPROVE SR-104`
- `DECLINE SR-104`

Use a short identifier associated with the current proposal version.

Do not invent a cost, arrival time, or guaranteed repair result. Approval for an inspection should not silently become authorization for unrelated work.

The dashboard should display the same request.

9. Processing owner decisions

Before accepting a reply:

1. Confirm the sender matches the configured owner.
2. Match the conversation and request identifier.
3. Confirm that the request is still pending.
4. Confirm that the proposal has not changed.
5. Submit the decision to MVP 2's existing approval function.
6. Store the message/event reference.
7. Resume the agent only after the backend accepts the decision.

Handle these cases explicitly:

Situation	Behavior
Valid approval	Record approval and allow the workflow to continue.
Valid decline	Record decline; do not contact a technician.
Missing request ID with multiple pending requests	Ask which request the owner means.
Unknown sender	Do not authorize work.
Superseded proposal	Ask the owner to review the current request.
Repeated approval	Return the existing result without another call.
No reply	Leave approval pending.

Use deterministic parsing for the explicit approval commands. Nemotron can interpret ordinary clarification text, but ambiguous wording should not grant permission.

10. Technician-call authorization

Before starting the call, the calling adapter should receive or retrieve:

- Communication action ID.
- Incident ID.
- Work-order ID.
- Approved service-request ID and version.
- Technician ID and allowed contact reference.
- Approved work scope.
- Spoken work summary.

Immediately before calling, the backend must verify that:

- Approval remains valid.
- The work order is active.
- The recipient is the assigned technician.
- The requested work matches the approval.
- No equivalent call is already in progress or completed.

The model’s request to call is not itself authorization.

11. Simplest complete technician call

Use an automated notification call with a short spoken summary.

The content should identify:

1. The automated maintenance assistant.
2. The work-order identifier.
3. The building.
4. The observed issue.
5. The requested inspection.
6. That the owner approved the visit.
7. How to accept or decline.

For the demonstration, identify the location as the appropriate building model rather than inventing a real property address.

Nemotron can prepare the structured summary. The voice component renders it as audio.

Recommended acceptance mechanism: after hearing the call, the technician replies through the configured text channel:

- `ACCEPT WO-104`
- `DECLINE WO-104`

This avoids requiring speech recognition for the first complete version.

If the chosen voice runner supports keypad input, a carefully mapped accept/decline option is another possibility—but verify that support before promising it.

12. Interactive voice is a separate extension

A technician having a live conversation with Nemotron requires additional pieces:

Component	Purpose
Phone connection	Connect the technician through Photon/SIP.
Audio handling	Send and receive call audio.
Speech recognition	Convert spoken replies into text, where required.
Nemotron	Interpret the conversation and choose tool actions.
Speech generation	Produce audible responses.
Conversation state	Keep the call tied to the work order.

Do not assume the messaging SDK provides this entire pipeline.

For the deadline, complete **notification call + explicit text acceptance** first. It still demonstrates an automated owner-approved service workflow.

13. Call progress and outcomes

Track the telephone connection separately from technician acceptance.

Call outcome	What it establishes
Call requested	An attempt was queued.
Dialing/ringing	The calling system is attempting connection.
Connected	A call connection exists; it does not establish human acceptance.
Completed	The call ended.
Busy/no answer	The attempt did not produce a confirmed response.
Failed	The calling system reported a failure.
Unknown	The result could not be established reliably.

A connection could reach voicemail or another person. Do not mark the work order accepted without an explicit response from the expected technician.

If no one accepts, leave the dashboard at **Awaiting technician response** or a suitable failure state.

MVP 4 decides whether another eligible technician should be considered under the approved scope and configured contact policy. MVP 5 should not independently start calling arbitrary alternatives.

14. Technician progress and findings

After acceptance, support a small set of responses:

Reply	Backend effect
`ACCEPT WO-104`	Record acceptance.
`DECLINE WO-104`	Record refusal.

`START WO-104`	Record that work has begun.
`REPORT WO-104 …`	Store inspection or repair findings.

Preserve the technician's original text. Nemotron can extract structured findings, but ambiguous statements should remain uncertain.

A report such as "connection reseated; lights restored" can become:

- Reported cause.
- Action performed.
- Reported outcome.

It should not immediately become a verified repair case. MVP 2 and MVP 4 still handle verification.

Keep the owner updated at meaningful milestones, not after every internal operation.

15. Normalized events

Translate provider-specific messages and call results into your own event format.

Each event should include:

Field	Purpose
Internal event ID	Identify the recorded event.
Provider event/message/call reference	Deduplicate and investigate delivery.
Event type	Owner reply, technician reply, call connected, call failed, etc.
Sender or recipient identifier	Match the configured contact.
Incident/request/work-order reference	Route it to the correct workflow.
Timestamp	Record when it occurred or was received.
Content/result	Preserve the actual response.
Source	Live integration, test adapter, or replay.

Resolve roles using server-side records. Do not trust message text that claims “I am the owner.”

16. Receiving messages

For Stable Spectrum, you can receive messages through a long-running stream or HTTP webhooks. Choose one for the first version.

- A working SDK stream may be simplest for a continuously running local backend.
- Webhooks require a reachable endpoint and correct signature verification.

Photon documents HMAC verification for native Spectrum webhooks and requires the exact raw request bytes for verification. Don’t parse and reserialize the body before checking it. Webhook documentation

If using Express, configure the webhook route’s raw-body handling before a global JSON parser changes the request body.

After receiving an event:

1. Verify its origin through the selected integration.
2. Check whether it has already been processed.
3. Record it through the backend.
4. Route it to the relevant decision or work-order handler.
5. Resume the waiting agent workflow if appropriate.

Calling events may come from the SIP application rather than the messaging SDK. Authenticate that integration separately.

17. Duplicate prevention and restart behavior

This MVP must survive duplicate messages without repeatedly contacting people.

Use:

- Stable outbound action IDs.
- Provider-supported deduplication identifiers where available.
- Stored provider references.
- Processed inbound-event IDs.
- Explicit pending/sent/failed/unknown states.

Create and persist the action before sending.

If a timeout occurs after a call may already have started, check its outcome before retrying. Don’t assume timeout means nothing happened.

Photon's recovery guidance specifically recommends stable message identifiers and persisted progress to reduce duplicate sends. Use the mechanism exposed by the selected provider rather than assuming every channel behaves identically. Recovery guidance

On restart:

- Completed owner decisions remain.
- Sent communications remain recorded.
- Duplicate inbound events are ignored.
- Uncertain external actions are reviewed rather than blindly repeated.
- Pending approval does not become approved.

18. Suggested module boundaries

Module	Responsibility
`photonMessaging`	Send and receive through the selected text channel.
`voiceAdapter`	Initiate calls and expose their results.
`communicationActions`	Validate requests and reference backend action records.
`ownerReplyHandler`	Route clarification and approval responses.
`technicianReplyHandler`	Route acceptance, progress, and findings.
`eventNormalizer`	Convert provider events into the application format.
`communicationAudit`	Record attempts, references, and failures.

These are proposed local modules. They are not Photon SDK classes or methods.

Keep workflow rules in MVP 2 and reasoning in MVP 4 rather than duplicating them here.

19. Configuration

The integration needs protected configuration for:

- Selected Photon platform/version.
- Project credentials.
- Messaging provider and sending line.
- Calling line and SIP credentials.

- Owner contact.
- Technician contacts.
- Webhook signing secret, if applicable.
- Voice-runner connection.
- Timeouts and bounded retry policy.
- Live versus test mode.

Keep real secrets and contact details out of the frontend bundle and public repository.

In test mode, restrict communications to the registered teammate contacts. A test adapter must be visibly distinguished from a live send or call.

20. Build order

Complete the riskiest connections first:

1. Identify the Photon project version and available line.
2. Send one real text to the owner.
3. Receive one real reply.
4. Place one test call with audible content.
5. Make that call programmatically from the calling adapter.
6. Connect owner replies to backend approval.
7. Verify that unapproved calls are rejected.
8. Trigger the call from an approved work order.
9. Receive explicit technician acceptance.
10. Receive repair findings.
11. Update dashboard state through the backend.
12. Add deduplication, restart recovery, and failure handling.

Do not leave the first phone-call test until the end of integration.

21. Acceptance tests

Test	Expected result
Owner clarification sent	Reply resumes the correct investigation.
Owner answers “yes” to a sensor question	No service approval is inferred.
Owner approves a valid request	Backend records the matching decision.
Owner declines	No technician call occurs.
Unknown contact sends approval	Authorization is rejected.

Two requests are pending	Reply updates only the identified request.
Approval arrives through dashboard	Same backend workflow can initiate contact.
Approval message arrives twice	One intended call action exists.
Agent requests an unapproved call	Backend blocks it.
Call connects	Work order still awaits explicit acceptance.
Technician accepts by text	Correct work order becomes accepted.
Technician does not answer	Dashboard does not claim acceptance.
Technician reports completion	Verification remains pending.
Provider event is duplicated	No repeated side effect.
Network result is uncertain	No blind duplicate call.
Application restarts	Records and pending decisions survive.
Test adapter is used	UI/logs identify the action as simulated.

22. Handoff deliverables

The MVP 5 developer should provide:

- Confirmed Photon platform/version and enabled channel.
- Working real text send-and-reply integration.
- Working automated outbound notification call.
- Explicit technician acceptance handling.
- Owner decision correlation.
- Technician progress/findings handling.
- Normalized events for the backend.
- Communication-action references and duplicate prevention.
- Failure and restart behavior.
- Configuration template without real secrets.
- Setup instructions for the calling runner.
- Test fixtures and a clearly labeled fallback demonstration.

MVP 5 is complete when an owner receives a real approval text, their authorized decision reaches the backend, an approved work order triggers an automated technician call, and the technician's explicit response updates the same work order on the dashboard.

A full spoken conversation with Nemotron can follow after that loop works reliably.